

PRODUCT REQUIREMENTS DOCUMENT

iMosque SafarRoute

Shortest Path and Multi-Objective Mosque Routing for Safar Mode

Versi Dokumen: 1.0

Status: Draft Final untuk Proyek UAS

Mata Kuliah: Big Data Analysis

Universitas: Universitas Nusa Mandiri

Kelompok: Kelompok 2 – Shortest Path & Multi-Objective Routing

Anggota Tim:

1. Riki Daniel Tanebeth – 12114240032
 2. Adhitya Januarizky Hadi Putra – 121514240045
 3. Muhammad Yusuf – 121614240047
-

1. Ringkasan Produk

iMosque SafarRoute adalah aplikasi prototipe untuk membantu pengguna dalam perjalanan menemukan masjid dan menentukan rute terbaik menuju masjid tersebut.

Berbeda dari aplikasi navigasi biasa yang hanya mencari rute berdasarkan jarak atau waktu tercepat, SafarRoute menggunakan pendekatan **multi-objective routing**. Sistem mempertimbangkan beberapa faktor secara bersamaan, yaitu:

- waktu tempuh;
- jarak perjalanan;
- kedekatan dengan waktu salat;
- estimasi keterlambatan tiba;
- kapasitas atau atribut masjid, apabila data tersedia;
- estimasi biaya perjalanan;
- preferensi pengguna.

Sistem menggunakan data masjid iMosque dan jaringan jalan dari sumber terbuka, seperti OpenStreetMap. Algoritma utama yang dibandingkan adalah **Dijkstra** dan **A***.

Produk dikembangkan sebagai modul untuk fitur **Safar Mode** pada aplikasi iMosque dan menyediakan REST API agar dapat diintegrasikan dengan sistem kelompok lain.

2. Latar Belakang

Pengguna yang sedang melakukan perjalanan sering menghadapi beberapa masalah ketika waktu salat hampir tiba:

1. Tidak mengetahui masjid yang dapat dicapai sebelum atau tidak lama setelah waktu salat.
2. Aplikasi navigasi umum biasanya hanya mengoptimalkan waktu atau jarak.
3. Masjid terdekat secara geografis belum tentu menjadi pilihan terbaik karena akses jalan dapat lebih panjang.
4. Informasi waktu salat, rute, jarak, dan karakteristik masjid berada pada sumber yang berbeda.
5. Penggunaan layanan peta komersial dapat meningkatkan biaya operasional ketika jumlah pengguna dan data semakin besar.

iMosque memiliki potensi data sekitar 1,2 juta masjid di Indonesia. Proyek UAS menggunakan subset sekitar 50.000 masjid sebagai data awal. Oleh sebab itu, sistem routing harus dirancang agar mampu bekerja pada data besar, dapat dievaluasi skalabilitasnya, dan tidak bergantung pada Google Maps.

3. Pernyataan Masalah

Sistem perlu menjawab pertanyaan utama berikut:

“Masjid mana yang paling sesuai untuk didatangi oleh pengguna dalam perjalanan, dan rute mana yang memberikan kombinasi terbaik antara waktu tempuh, jarak, waktu salat, biaya, dan atribut masjid?”

Masalah tersebut tidak cukup diselesaikan dengan mencari masjid yang memiliki jarak garis lurus paling dekat. Sistem perlu:

1. menemukan kandidat masjid berdasarkan jaringan jalan;
 2. menghitung rute dari lokasi pengguna;
 3. memperkirakan waktu tiba;
 4. membandingkan waktu tiba dengan waktu salat;
 5. memberikan peringkat terhadap beberapa alternatif;
 6. menjelaskan alasan suatu rute direkomendasikan.
-

4. Visi Produk

Membangun layanan routing masjid yang terbuka, hemat biaya, dapat dijelaskan, dan mampu dikembangkan dari dataset 50.000 masjid menuju skala nasional.

5. Tujuan Produk

5.1 Tujuan Utama

1. Mengimplementasikan algoritma Dijkstra dan A* pada graph jaringan jalan dan masjid.
2. Menghasilkan rekomendasi rute menuju masjid tanpa menggunakan Google Maps API.
3. Mengembangkan metode multi-objective routing untuk Safar Mode.
4. Menyediakan REST API yang dapat digunakan oleh aplikasi iMosque atau Kelompok 4.
5. Membandingkan performa Dijkstra dan A*.
6. Mengevaluasi sistem berdasarkan akurasi, latency, penggunaan resource, dan skalabilitas.
7. Menunjukkan kemungkinan implementasi pada data masjid berskala besar.

5.2 Tujuan Akademik

1. Menerapkan konsep Big Graphs.
 2. Menguji algoritma graph pada dataset besar.
 3. Menghasilkan eksperimen yang dapat direplikasi.
 4. Menyediakan analisis teknis, bukan hanya demonstrasi antarmuka.
 5. Menghasilkan unsur kebaruan yang relevan dengan Safar Mode.
-

6. Sasaran Pengguna

6.1 Musafir

Pengguna yang sedang melakukan perjalanan antarkota maupun perjalanan jarak pendek dan perlu menemukan masjid di sepanjang rute.

Kebutuhan utama:

- menemukan masjid yang masih dapat dicapai sebelum salat;
- mendapatkan rute cepat dan mudah;
- membandingkan beberapa alternatif masjid;
- mengetahui alasan rekomendasi.

6.2 Pengguna Harian

Pengguna yang berada di lokasi yang belum dikenal dan ingin menemukan masjid berdasarkan preferensi jarak, waktu, atau fasilitas.

6.3 Pengembang iMosque

Pengembang yang memerlukan layanan routing melalui API untuk diintegrasikan dengan Safar Mode.

6.4 Peneliti atau Administrator

Pengguna teknis yang ingin melihat hasil benchmark algoritma, latency, penggunaan resource, dan pengaruh ukuran data.

7. Ruang Lingkup

7.1 Ruang Lingkup MVP

Versi MVP wajib menyediakan:

1. Input titik asal berupa latitude dan longitude.
2. Input waktu keberangkatan.
3. Pemilihan jenis salat.
4. Pencarian kandidat masjid dalam radius tertentu.
5. Perhitungan rute menggunakan Dijkstra.
6. Perhitungan rute menggunakan A*.
7. Perbandingan waktu eksekusi kedua algoritma.
8. Rekomendasi minimal tiga masjid.
9. Estimasi jarak dan waktu tempuh.
10. Status kemungkinan tiba sebelum atau setelah waktu salat.
11. Skor multi-objective.
12. Visualisasi rute sederhana.
13. REST API.
14. Pencatatan hasil pengujian.

7.2 Ruang Lingkup Fitur Lanjutan

Fitur lanjutan yang dikerjakan setelah MVP stabil:

- pengaturan bobot preferensi pengguna;
- rekomendasi masjid di sepanjang koridor perjalanan;
- estimasi biaya bahan bakar atau tol;
- filter fasilitas masjid;
- integrasi data waktu salat real-time;
- alternatif rute;
- caching hasil routing;
- integrasi dengan stream processing dari Kelompok 3;
- integrasi end-to-end oleh Kelompok 4.

7.3 Di Luar Ruang Lingkup

Hal-hal berikut tidak menjadi kewajiban utama Kelompok 2:

- pembangunan graph database nasional dari awal;
- prediksi lalu lintas real-time;
- prediksi kepadatan masjid;
- community detection;
- benchmark seluruh jenis graph database;
- aplikasi mobile production-ready;
- sistem autentikasi pengguna lengkap;
- pembayaran atau donasi;
- navigasi turn-by-turn setara aplikasi komersial.

Fondasi graph diharapkan dapat memanfaatkan keluaran Kelompok 1 dan Kelompok 6.

8. Nilai Kebaruan Produk

Nilai kebaruan SafarRoute terletak pada kombinasi berikut:

8.1 Prayer-Aware Routing

Rute tidak hanya dinilai dari jarak, tetapi juga dari kemungkinan pengguna tiba sebelum atau mendekati waktu salat.

8.2 Multi-Objective Mosque Selection

Sistem memilih kombinasi masjid dan rute terbaik, bukan sekadar menghitung rute menuju masjid yang telah ditentukan.

8.3 Explainable Recommendation

Setiap rekomendasi disertai penjelasan, misalnya:

“Masjid A direkomendasikan karena dapat dicapai sekitar 8 menit sebelum Magrib, memiliki tambahan jarak hanya 1,2 km, dan skor total lebih baik dibandingkan dua alternatif lainnya.”

8.4 Open-Source and Cost-Efficient Routing

Sistem menggunakan data dan teknologi terbuka sehingga tidak bergantung pada biaya per permintaan dari penyedia peta komersial.

8.5 Scalable Graph-Based Architecture

Model graph dirancang agar dapat diuji pada subset 50.000 masjid dan secara konseptual dapat dikembangkan menuju dataset penuh.

9. Skenario Pengguna Utama

9.1 Mencari Masjid Sebelum Waktu Salat

1. Pengguna mengirim lokasi saat ini.
2. Pengguna memilih waktu salat yang dituju.
3. Sistem mengambil waktu salat pada wilayah pengguna.
4. Sistem menemukan masjid dalam radius pencarian.
5. Sistem menghitung rute menuju setiap kandidat.
6. Sistem memperkirakan waktu tiba.
7. Sistem menghitung skor multi-objective.
8. Sistem menampilkan tiga rekomendasi terbaik.

9.2 Membandingkan Rute

1. Pengguna memilih satu masjid.
2. Sistem menghitung rute menggunakan Dijkstra dan A*.
3. Sistem menampilkan jarak, estimasi waktu, jumlah node yang diperiksa, dan waktu komputasi.
4. Pengguna dapat memilih rute yang direkomendasikan.

9.3 Mencari Masjid di Sepanjang Perjalanan

1. Pengguna memasukkan titik asal dan tujuan akhir.
 2. Sistem membentuk koridor perjalanan.
 3. Sistem mencari masjid yang berada dekat koridor tersebut.
 4. Sistem menghitung tambahan jarak atau detour.
 5. Sistem memberi peringkat berdasarkan waktu tiba, detour, dan waktu salat.
-

10. Kebutuhan Fungsional

ID	Kebutuhan	Prioritas
FR-01	Sistem menerima lokasi asal dalam bentuk latitude dan longitude.	Wajib
FR-02	Sistem menerima tujuan akhir perjalanan secara opsional.	Tinggi
FR-03	Sistem menerima waktu keberangkatan.	Wajib

ID	Kebutuhan	Prioritas
FR-04	Sistem menerima jenis salat yang menjadi target.	Wajib
FR-05	Sistem menemukan kandidat masjid dalam radius yang dapat dikonfigurasi.	Wajib
FR-06	Sistem menghubungkan lokasi pengguna ke node jalan terdekat.	Wajib
FR-07	Sistem menghitung shortest path menggunakan Dijkstra.	Wajib
FR-08	Sistem menghitung shortest path menggunakan A*.	Wajib
FR-09	Sistem menghitung estimasi jarak dan waktu tempuh.	Wajib
FR-10	Sistem menghitung perkiraan waktu tiba.	Wajib
FR-11	Sistem membandingkan waktu tiba dengan waktu salat.	Wajib
FR-12	Sistem menghasilkan skor multi-objective untuk setiap kandidat.	Wajib
FR-13	Sistem mengurutkan kandidat berdasarkan skor terbaik.	Wajib
FR-14	Sistem mengembalikan minimal tiga rekomendasi apabila kandidat tersedia.	Wajib
FR-15	Sistem menyediakan penjelasan alasan rekomendasi.	Tinggi
FR-16	Sistem menampilkan geometri rute pada peta sederhana.	Tinggi
FR-17	Sistem menyediakan filter fasilitas apabila atribut tersedia.	Menengah
FR-18	Sistem menyediakan pilihan profil routing.	Tinggi
FR-19	Sistem menyediakan endpoint pemeriksaan kesehatan layanan.	Wajib
FR-20	Sistem mencatat latency dan algoritma yang digunakan.	Wajib
FR-21	Sistem dapat mengembalikan hasil benchmark Dijkstra dan A*.	Wajib
FR-22	Sistem memberikan pesan yang jelas ketika rute tidak ditemukan.	Wajib
FR-23	Sistem menyediakan API yang dapat digunakan Kelompok 4.	Wajib
FR-24	Sistem mendukung penyesuaian bobot objektif melalui parameter API.	Tinggi

11. Profil Routing

Sistem menyediakan beberapa profil awal.

11.1 Fastest

Memprioritaskan waktu tempuh tercepat.

Contoh bobot:

- waktu tempuh: 60%;
- kedekatan waktu salat: 25%;
- jarak: 10%;
- biaya: 5%.

11.2 Prayer Priority

Memprioritaskan kemungkinan tiba sebelum waktu salat.

Contoh bobot:

- kedekatan waktu salat: 50%;
- waktu tempuh: 30%;
- jarak: 10%;
- biaya: 10%.

11.3 Low Cost

Memprioritaskan rute dengan estimasi biaya terendah.

Contoh bobot:

- biaya perjalanan: 45%;
- jarak: 25%;
- waktu tempuh: 20%;
- kedekatan waktu salat: 10%.

11.4 Balanced

Memberikan keseimbangan terhadap seluruh kriteria.

Bobot tetap harus dapat diubah selama eksperimen. Total bobot harus bernilai 1 atau 100%.

12. Model Multi-Objective Routing

Untuk setiap kombinasi rute dan masjid, sistem menghitung nilai:

$$Score = w_t T + w_d D + w_p P + w_b B + w_f F$$

Keterangan:

- T : nilai normalisasi waktu tempuh;

- D : nilai normalisasi jarak;
- P : penalti terhadap waktu salat;
- B : nilai normalisasi estimasi biaya;
- F : penalti atribut atau fasilitas;
- w : bobot setiap objektif.

Sistem mencari kandidat dengan nilai skor paling kecil.

12.1 Penalti Waktu Salat

Contoh aturan:

Kondisi waktu tiba	Penalti
Tiba 5–20 menit sebelum waktu salat	0,00
Tiba lebih dari 20 menit sebelumnya	0,10
Tiba 0–5 menit sebelum waktu salat	0,15
Tiba maksimal 10 menit setelah waktu salat	0,50
Tiba lebih dari 10 menit setelah waktu salat	1,00

Aturan ini merupakan parameter eksperimen dan bukan ketentuan keagamaan. Waktu iqamah tidak boleh diasumsikan tersedia apabila dataset tidak memuatnya.

12.2 Estimasi Biaya

Apabila tidak tersedia data tarif yang lengkap, biaya dihitung sebagai estimasi:

$$Biaya = Jarak \times Konsumsi \text{ Bahan Bakar} \times Harga \text{ Bahan Bakar}$$

Biaya tol hanya dimasukkan apabila informasi jalan tol dapat diperoleh secara valid. Sistem harus memberi label bahwa nilai tersebut adalah **estimasi**, bukan biaya aktual.

13. Persyaratan Data

13.1 Data Masjid

Atribut minimal:

Atribut	Keterangan
mosque_id	ID unik masjid

Atribut	Keterangan
name	Nama masjid
latitude	Latitude
longitude	Longitude
address	Alamat, apabila tersedia
city	Kota atau kabupaten
province	Provinsi
capacity	Kapasitas, apabila tersedia
facilities	Fasilitas, apabila tersedia
source	Sumber data
updated_at	Tanggal pembaruan data

Atribut yang kosong tidak boleh diisi dengan nilai yang tidak memiliki dasar. Sistem harus dapat beroperasi hanya dengan ID, nama, latitude, dan longitude.

13.2 Data Jaringan Jalan

Sumber utama yang disarankan adalah OpenStreetMap.

Atribut edge jalan:

- source node;
- target node;
- panjang jalan;
- tipe jalan;
- arah jalan;
- estimasi kecepatan;
- estimasi waktu tempuh;
- informasi tol apabila tersedia.

13.3 Data Waktu Salat

Data waktu salat dapat diperoleh melalui:

- Prayer Times API seperti AlAdhan;
- library perhitungan waktu salat;
- keluaran stream processing Kelompok 3.

Sumber dan metode perhitungan harus dicatat agar eksperimen dapat direplikasi.

13.4 Kerahasiaan Dataset

Dataset iMosque hanya digunakan untuk kebutuhan tugas kuliah.

Ketentuan yang harus diterapkan:

1. Dataset mentah tidak dimasukkan ke repository publik.
 2. Repository hanya menyertakan sample data anonim atau data sintetis.
 3. Lokasi file dataset dikonfigurasi melalui environment variable.
 4. README menjelaskan prosedur memperoleh atau menempatkan dataset tanpa membagikan data rahasia.
 5. Log tidak menyimpan seluruh isi data masjid.
-

14. Model Graph

14.1 Node

Mosque

Merepresentasikan masjid.

Properti utama:

- mosque_id;
- name;
- latitude;
- longitude;
- capacity;
- facilities.

RoadNode

Merepresentasikan persimpangan atau titik pada jaringan jalan.

Properti utama:

- node_id;
- latitude;
- longitude.

14.2 Edge

ROAD_SEGMENT

Menghubungkan RoadNode dengan RoadNode.

Properti:

- length_meter;
- travel_time_second;
- road_type;
- max_speed;
- one_way;
- toll.

ACCESS_TO

Menghubungkan Mosque dengan RoadNode terdekat.

Properti:

- access_distance_meter;
- access_time_second.

14.3 Node Lokasi Pengguna

Lokasi pengguna sebaiknya diperlakukan sebagai data sementara dan tidak perlu disimpan permanen dalam graph. Lokasi tersebut dipetakan ke RoadNode terdekat saat permintaan diterima.

15. Algoritma Routing

15.1 Dijkstra

Dijkstra digunakan sebagai baseline karena dapat menemukan shortest path pada graph berbobot non-negatif.

Keluaran pengujian:

- total jarak;
- total waktu;
- jumlah node yang dieksplorasi;
- waktu komputasi;
- penggunaan memori.

15.2 A*

A* menggunakan fungsi heuristik berdasarkan jarak geografis menuju tujuan.

Heuristik harus bersifat admissible agar tidak melebihi biaya sebenarnya. Salah satu pendekatan adalah menggunakan jarak Haversine dibagi kecepatan maksimum yang diizinkan.

15.3 Alur Perhitungan

1. Validasi input.
2. Cari node jalan terdekat dari lokasi pengguna.
3. Lakukan spatial filtering kandidat masjid.
4. Petakan setiap masjid ke node aksesnya.
5. Hitung rute menggunakan algoritma yang dipilih.
6. Hitung estimasi waktu tiba.
7. Ambil waktu salat.
8. Hitung seluruh nilai objektif.
9. Normalisasi nilai.
10. Hitung skor total.
11. Urutkan rekomendasi.
12. Kembalikan hasil dan penjelasan.

15.4 Optimasi Pencarian Kandidat

Sistem tidak harus menjalankan shortest path menuju seluruh 50.000 masjid.

Tahap kandidat dapat menggunakan:

- bounding box;
- geospatial index;
- radius Haversine;
- pembatasan jumlah kandidat terdekat;
- koridor perjalanan;
- pencarian bertahap dengan radius yang diperbesar.

Contoh:

1. Cari maksimal 20 masjid dalam radius 5 km.
 2. Apabila tidak ditemukan, perluas menjadi 10 km.
 3. Jalankan routing detail hanya pada kandidat hasil filtering.
-

16. Spesifikasi API

16.1 Health Check

Endpoint

GET /api/v1/health

Respons

```
{
  "status": "healthy",
  "graph_status": "connected",
  "version": "1.0.0"
}
```

16.2 Mencari Rekomendasi Masjid

Endpoint

POST /api/v1/routes/recommend

Request

```
{
  "origin": {
    "latitude": -6.2000,
    "longitude": 106.8166
  },
  "departure_time": "2026-07-11T17:10:00+07:00",
  "prayer": "maghrib",
  "algorithm": "astar",
  "profile": "prayer_priority",
  "search_radius_km": 10,
  "maximum_results": 3
}
```

Respons

```
{
  "request_id": "route-001",
  "prayer": {
```

```

    "name": "maghrib",
    "time": "17:55:00",
    "timezone": "Asia/Jakarta"
  },
  "recommendations": [
    {
      "rank": 1,
      "mosque": {
        "id": "M-1001",
        "name": "Masjid Contoh",
        "latitude": -6.2100,
        "longitude": 106.8200
      },
      "route": {
        "distance_km": 4.8,
        "estimated_duration_minute": 18,
        "estimated_arrival": "2026-07-11T17:28:00+07:00"
      },
      "arrival_status": "before_prayer",
      "minutes_before_prayer": 27,
      "score": 0.184,
      "explanation": "Waktu tiba sesuai dan tambahan jarak relatif rendah."
    }
  ],
  "processing_time_ms": 420
}

```

16.3 Membandingkan Algoritma

Endpoint

POST /api/v1/routes/benchmark

Keluaran:

- hasil Dijkstra;
- hasil A*;
- perbedaan jarak;
- perbedaan waktu komputasi;
- jumlah node yang dikunjungi;
- kesimpulan algoritma yang lebih efisien untuk kasus tersebut.

16.4 Mendapatkan Geometri Rute

Endpoint

```
GET /api/v1/routes/{route_id}
```

Keluaran dapat menggunakan format GeoJSON agar mudah ditampilkan pada Leaflet.

16.5 Mendapatkan Daftar Profil

Endpoint

```
GET /api/v1/routing-profiles
```

17. Antarmuka Pengguna

Aplikasi demo berbasis web minimal memiliki empat bagian.

17.1 Form Pencarian

Komponen:

- lokasi asal;
- tujuan akhir opsional;
- waktu keberangkatan;
- pilihan salat;
- pilihan algoritma;
- pilihan profil;
- radius pencarian;
- tombol cari.

17.2 Peta

Peta menampilkan:

- lokasi pengguna;
- kandidat masjid;
- rute rekomendasi;
- rute alternatif;
- popup informasi masjid.

Leaflet dapat digunakan sebagai library visualisasi dengan tile map yang sesuai ketentuan penyedia.

17.3 Panel Rekomendasi

Untuk setiap rekomendasi ditampilkan:

- nama masjid;

- jarak;
- estimasi durasi;
- waktu tiba;
- selisih terhadap waktu salat;
- estimasi biaya;
- skor;
- alasan rekomendasi.

17.4 Panel Benchmark

Panel teknis menampilkan:

- algoritma;
- execution time;
- explored nodes;
- route distance;
- memory usage;
- ukuran graph yang diuji.

18. Kebutuhan Nonfungsional

18.1 Performa

Target awal pada lingkungan pengujian yang didokumentasikan:

Metrik	Target MVP
P95 latency rekomendasi	Maksimal 3 detik
P95 health check	Maksimal 300 ms
API success rate pada test case valid	Minimal 95%
Selisih biaya rute A* dan Dijkstra	0 untuk bobot yang sama
Kandidat awal	Maksimal 20–50 masjid per request

Target tersebut harus dinilai sebagai sasaran pengujian, bukan klaim hasil sebelum implementasi.

18.2 Skalabilitas

Pengujian dilakukan pada beberapa ukuran data, misalnya:

- 1.000 masjid;
- 5.000 masjid;
- 10.000 masjid;

- 25.000 masjid;
- 50.000 masjid.

Apabila sumber daya memungkinkan, pengujian dapat diperluas menggunakan data sintetis atau dataset penuh melalui koordinasi dengan kelompok lain.

18.3 Keandalan

- Sistem tidak berhenti ketika salah satu kandidat tidak memiliki rute.
- Kandidat yang gagal dihitung dicatat dan dilewati.
- API memberikan status error yang konsisten.
- Kegagalan layanan waktu salat memiliki fallback yang terdokumentasi.
- Query memiliki timeout.

18.4 Maintainability

- Kode dipisahkan menjadi modul data, graph, routing, scoring, API, dan evaluation.
- Konfigurasi tidak ditulis langsung di source code.
- Fungsi utama memiliki unit test.
- API didokumentasikan menggunakan OpenAPI atau Swagger.
- Repository memiliki petunjuk instalasi yang dapat direplikasi.

18.5 Portabilitas

Sistem dapat dijalankan menggunakan Docker Compose pada komputer lokal tanpa layanan berbayar.

19. Privasi dan Keamanan

1. Lokasi pengguna tidak disimpan permanen secara default.
 2. Log hanya menyimpan koordinat yang telah dikurangi presisinya atau request ID.
 3. API memvalidasi rentang latitude dan longitude.
 4. Parameter radius dan jumlah kandidat dibatasi.
 5. Query graph menggunakan parameter binding.
 6. Kredensial database disimpan melalui environment variable.
 7. Endpoint dokumentasi dapat dibatasi pada lingkungan demo.
 8. Dataset rahasia tidak disertakan dalam image Docker atau GitHub.
-

20. Arsitektur Produk yang Direkomendasikan

20.1 Komponen

Web Demo

Teknologi yang dapat digunakan:

- React, Vue, atau HTML sederhana;
- Leaflet;
- OpenStreetMap.

Routing API

Teknologi yang direkomendasikan:

- Python;
- FastAPI;
- Pydantic;
- Uvicorn.

Routing Engine

Pilihan implementasi:

- NetworkX atau OSMnx untuk baseline;
- Neo4j dengan Cypher atau Graph Data Science apabila fondasi tersedia;
- custom implementation untuk Dijkstra dan A* agar technical depth dapat ditunjukkan.

Graph Storage

Neo4j Community Edition dapat digunakan untuk MVP karena relatif mudah disiapkan dan diintegrasikan. Penggunaan graph database lain dapat menyesuaikan hasil Kelompok 1 dan Kelompok 6.

Cache

Redis bersifat opsional. Untuk MVP, cache in-memory dapat digunakan terlebih dahulu.

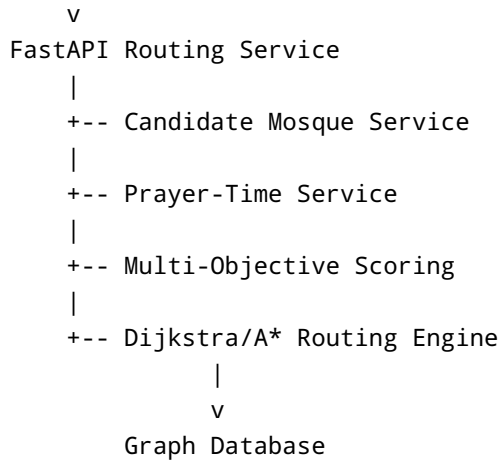
Prayer-Time Service

Layanan ini mengakses API waktu salat atau library perhitungan lokal.

20.2 Alur Data

Web Client

|



21. Strategi Pengujian

21.1 Unit Testing

Mencakup:

- perhitungan Haversine;
- normalisasi skor;
- validasi bobot;
- penalti waktu salat;
- estimasi waktu tiba;
- perhitungan biaya;
- Dijkstra pada graph kecil;
- A* pada graph kecil.

21.2 Integration Testing

Mencakup:

- API ke graph database;
- API ke prayer-time service;
- kandidat masjid ke routing engine;
- routing engine ke scoring;
- serialisasi hasil GeoJSON.

21.3 Correctness Testing

Dijkstra dan A* dijalankan pada pasangan origin-destination yang sama.

Kriteria:

- total bobot rute harus sama apabila keduanya menggunakan cost function yang sama;
- rute harus hanya melewati edge yang valid;
- rute harus mematuhi arah jalan;
- estimasi waktu merupakan penjumlahan biaya edge.

21.4 Performance Testing

Variabel eksperimen:

- jumlah node;
- jumlah edge;
- jumlah masjid;
- jumlah kandidat;
- algoritma;
- radius pencarian;
- penggunaan cache;
- jumlah concurrent request.

21.5 Scalability Testing

Eksperimen mencatat:

- waktu import;
- ukuran penyimpanan;
- waktu query kandidat;
- waktu routing;
- throughput;
- penggunaan CPU;
- penggunaan memori.

21.6 Failure Testing

Kasus yang diuji:

- koordinat di luar Indonesia;
- koordinat tidak valid;
- tidak ada masjid dalam radius;
- tidak ada rute;
- database tidak tersedia;
- API waktu salat gagal;
- bobot tidak berjumlah 100%;
- radius terlalu besar;
- waktu keberangkatan tidak valid.

22. Metrik Evaluasi

22.1 Metrik Algoritma

- execution time;
- explored nodes;
- path cost;
- path distance;
- peak memory;
- preprocessing time.

22.2 Metrik Sistem

- average latency;
- P50 latency;
- P95 latency;
- P99 latency;
- throughput;
- error rate;
- cache hit rate.

22.3 Metrik Kualitas Rekomendasi

- persentase rekomendasi yang memungkinkan tiba sebelum waktu salat;
- rata-rata tambahan jarak;
- rata-rata selisih waktu tiba terhadap waktu salat;
- stabilitas peringkat ketika bobot berubah;
- jumlah kasus tanpa kandidat;
- perbandingan dengan baseline masjid terdekat berdasarkan jarak garis lurus.

22.4 Baseline Eksperimen

Sistem dibandingkan dengan:

1. masjid dengan jarak Haversine terdekat;
 2. masjid dengan shortest travel time;
 3. multi-objective recommendation;
 4. Dijkstra dibandingkan A*.
-

23. Hipotesis Pengujian

H1

A* memiliki waktu eksekusi lebih rendah dan mengeksplorasi lebih sedikit node dibandingkan Dijkstra pada graph jalan yang sama.

H2

Masjid dengan jarak geografis terdekat tidak selalu menghasilkan waktu tempuh tercepat.

H3

Multi-objective routing menghasilkan rekomendasi yang lebih sesuai terhadap waktu salat dibandingkan routing berbasis jarak saja.

H4

Spatial pre-filtering mengurangi latency secara signifikan tanpa menghilangkan kandidat terbaik pada sebagian besar test case.

Hipotesis harus dibuktikan atau ditolak berdasarkan hasil eksperimen. Laporan tidak boleh menyatakan hipotesis sebagai fakta sebelum pengujian selesai.

24. Kriteria Penerimaan MVP

MVP dinyatakan selesai apabila:

1. Dataset masjid berhasil dimuat.
2. Graph jalan untuk wilayah uji berhasil dibangun.
3. Minimal 50.000 data masjid dapat diproses pada tahap pencarian kandidat atau import.
4. Dijkstra berhasil menemukan rute pada test case valid.
5. A* berhasil menemukan rute pada test case valid.
6. Hasil biaya shortest path Dijkstra dan A* konsisten.
7. Sistem menghasilkan minimal tiga rekomendasi ketika kandidat tersedia.
8. Setiap rekomendasi memuat jarak, waktu tempuh, waktu tiba, dan skor.
9. API dapat dipanggil melalui Swagger atau aplikasi demo.
10. Peta dapat menampilkan minimal satu rute.
11. Benchmark dapat diekspor ke CSV atau JSON.
12. Unit test fungsi kritis berhasil dijalankan.
13. README menjelaskan instalasi dan cara demo.
14. Dataset rahasia tidak masuk ke repository publik.
15. Kelompok 4 memperoleh dokumentasi API untuk integrasi.

25. Milestone Empat Pekan

Pekan 1 – Fondasi dan Proposal

Target:

- memahami struktur dataset;
- menentukan wilayah uji;
- membuat repository GitHub;
- menentukan graph schema;
- menyiapkan environment;
- memperoleh sample graph dari Kelompok 1 atau membuat graph sementara;
- menyusun proposal dan PRD;
- menentukan pembagian tugas.

Deliverable:

- proposal kelompok;
- PRD;
- pembagian tugas;
- repository;
- sample dataset;
- desain API awal.

Pekan 2 – MVP Routing

Target:

- preprocessing data;
- spatial candidate filtering;
- implementasi Dijkstra;
- implementasi A*;
- endpoint routing dasar;
- visualisasi rute;
- unit test awal.

Deliverable:

- MVP yang dapat didemonstrasikan;
- hasil shortest path;
- API awal;
- demo internal.

Pekan 3 – Multi-Objective dan Evaluasi Awal

Target:

- integrasi waktu salat;
- implementasi scoring;
- routing profile;
- endpoint recommendation;
- benchmark awal;
- integrasi awal dengan Kelompok 4;
- draft laporan.

Deliverable:

- fitur hampir lengkap;
- hasil evaluasi awal;
- dokumentasi API;
- draft laporan.

Pekan 4 – Finalisasi

Target:

- pengujian 50.000 data;
- performance testing;
- perbaikan error;
- Dockerisasi;
- finalisasi README;
- video demo;
- laporan 10–15 halaman;
- slide presentasi;
- integrasi akhir.

Deliverable:

- source code final;
 - dokumentasi;
 - hasil benchmark;
 - laporan;
 - video demo;
 - slide presentasi;
 - demo terintegrasi.
-

26. Pembagian Tugas Anggota

26.1 Riki Daniel Tanebeth – Project Leader dan Integrator

Tanggung jawab:

- mengelola scope dan milestone;
- koordinasi dengan Kelompok 1, 3, 4, dan 6;
- menyusun kontrak API;
- mengintegrasikan seluruh modul;
- memastikan repository dan dokumentasi rapi;
- menyusun laporan dan presentasi;
- membantu pengembangan backend.

26.2 Adhitya Januarizky Hadi Putra – Core Algorithm Developer

Tanggung jawab:

- implementasi Dijkstra;
- implementasi A*;
- implementasi heuristik;
- route reconstruction;
- unit testing algoritma;
- optimasi routing;
- analisis correctness.

26.3 Muhammad Yusuf – Data Engineer dan Evaluator

Tanggung jawab:

- preprocessing data masjid;
- pengolahan data OpenStreetMap;
- graph construction;
- spatial index;
- benchmark scalability;
- pengukuran CPU dan memori;
- visualisasi hasil evaluasi.

Pembagian tersebut dapat disesuaikan, tetapi setiap kontribusi perlu dicatat pada GitHub melalui commit, issue, atau pull request.

27. Ketergantungan Antarkelompok

Kelompok	Ketergantungan
Kelompok 1	Graph schema, node masjid, hubungan proximity atau akses jalan
Kelompok 3	Waktu salat atau stream data dinamis
Kelompok 4	Konsumen utama API routing dan integrator Safar Mode
Kelompok 6	Rekomendasi graph database dan hasil benchmark backend

Kelompok 2 harus tetap memiliki mock data atau fallback lokal agar pengembangan tidak berhenti ketika keluaran kelompok lain belum tersedia.

28. Risiko dan Mitigasi

Risiko	Dampak	Mitigasi
Data kapasitas atau fasilitas banyak yang kosong	Objektif tidak dapat dihitung	Jadikan atribut opsional dan jangan mengarang nilai
Graph kelompok lain terlambat	Routing tidak dapat dikembangkan	Gunakan graph OpenStreetMap wilayah kecil sebagai mock
Routing menuju banyak masjid lambat	Latency tinggi	Spatial filtering dan pembatasan kandidat
API waktu salat gagal	Rekomendasi tidak lengkap	Cache dan library perhitungan lokal
Dataset terlalu besar untuk laptop	Proses lambat atau kehabisan memori	Uji bertahap dan gunakan wilayah atau partisi
Heuristik A* tidak tepat	Hasil tidak optimal	Gunakan heuristik admissible dan bandingkan dengan Dijkstra
Waktu hanya empat pekan	Fitur tidak selesai	Prioritaskan MVP dan tunda fitur kosmetik
Ketergantungan layanan cloud	Demo berisiko gagal	Siapkan mode Docker lokal
Dataset rahasia masuk GitHub	Pelanggaran ketentuan	<code>.gitignore</code> , pre-commit check, dan sample sintetis
Klaim performa tidak didukung data	Nilai evaluasi menurun	Pisahkan target, hasil aktual, dan keterbatasan

29. Struktur Repository

```
imosque-safar-route/
├── README.md
├── LICENSE
├── .env.example
├── .gitignore
├── docker-compose.yml
├── docs/
│   ├── prd.md
│   ├── architecture.md
│   ├── api.md
│   └── evaluation.md
├── data/
│   ├── sample/
│   └── README.md
├── src/
│   ├── api/
│   ├── graph/
│   ├── routing/
│   ├── scoring/
│   ├── prayer_time/
│   ├── preprocessing/
│   └── evaluation/
├── frontend/
├── notebooks/
│   ├── data_exploration.ipynb
│   └── benchmark_analysis.ipynb
├── tests/
│   ├── unit/
│   └── integration/
├── scripts/
└── results/
    ├── benchmark/
    └── figures/
```

Dataset asli tidak ditempatkan di dalam repository.

30. Dokumentasi yang Wajib Tersedia

README minimal memuat:

1. deskripsi proyek;
 2. diagram arsitektur;
 3. teknologi yang digunakan;
 4. persyaratan sistem;
 5. cara instalasi;
 6. cara menyiapkan data;
 7. cara menjalankan aplikasi;
 8. cara menjalankan unit test;
 9. contoh API request;
 10. cara menjalankan benchmark;
 11. struktur repository;
 12. pembagian kontribusi anggota;
 13. batasan sistem;
 14. informasi kerahasiaan dataset.
-

31. Deliverable Akhir

Proyek menghasilkan:

1. Source code lengkap di GitHub.
 2. README dan dokumentasi API.
 3. Aplikasi demo routing.
 4. Implementasi Dijkstra dan A*.
 5. Multi-objective recommendation engine.
 6. Hasil benchmark.
 7. Dataset hasil eksperimen yang tidak melanggar kerahasiaan.
 8. Grafik evaluasi.
 9. Laporan proyek 10–15 halaman.
 10. Video demo 5–7 menit.
 11. Slide presentasi.
 12. Kontrak API untuk Kelompok 4.
-

32. Future Work

Pengembangan berikutnya dapat mencakup:

- traffic-aware dynamic routing;
- integrasi stream lokasi pengguna;

- prediksi kepadatan masjid;
- waktu iqamah berbasis data aktual;
- routing untuk kendaraan, sepeda, dan pejalan kaki;
- personalized recommendation;
- distributed graph processing;
- hierarchical routing;
- contraction hierarchies;
- live rerouting ketika kondisi berubah;
- integrasi penuh dengan aplikasi mobile iMosque;
- evaluasi terhadap dataset penuh 1,2 juta masjid.

33. Asumsi dan Batasan

1. Dataset awal berjumlah sekitar 50.000 masjid.
2. Data jaringan jalan diperoleh secara terpisah dari OpenStreetMap.
3. Atribut kapasitas dan fasilitas mungkin tidak lengkap.
4. Traffic real-time tidak termasuk kebutuhan wajib Kelompok 2.
5. Estimasi waktu tempuh bergantung pada panjang dan asumsi kecepatan jalan.
6. Estimasi biaya bukan biaya aktual.
7. Waktu salat mengikuti sumber atau metode perhitungan yang dipilih.
8. Sistem merupakan prototipe akademik, bukan layanan navigasi keselamatan production-ready.
9. Klaim skalabilitas harus dibatasi pada skala yang benar-benar diuji.
10. Pengembangan diprioritaskan untuk menyelesaikan MVP dalam empat pekan.

34. Definisi Selesai

Produk dianggap selesai ketika kode, dokumentasi, pengujian, dan demo dapat dijalankan ulang oleh dosen atau anggota kelompok lain menggunakan petunjuk repository, serta seluruh hasil evaluasi dapat ditelusuri ke konfigurasi eksperimen yang jelas.

Hasil akhir tidak hanya harus menunjukkan bahwa aplikasi dapat mencari rute, tetapi juga harus menjelaskan:

- mengapa rute tertentu dipilih;
- apakah A* lebih efisien dibandingkan Dijkstra;
- bagaimana multi-objective routing memengaruhi rekomendasi;
- bagaimana sistem menangani peningkatan ukuran data;
- apa keterbatasan yang masih ada.